Further simplification of the K215 proofs

Recent commit review and the next reusable lemmas

Research contribution for Vladimir Reshetnikov's ProveIt project

21 October 2026 UTC

Conclusion. The production baseline changed during this review. Phase five now uses an ordinary 2,240-row Lean source certificate with a shared decoder-parametric soundness proof and recorded incremental endpoint validation. The further 1,426-row normalized certificate remains a separately checked, unintegrated proposal. The best new runtime target is one return-to-dispatch scan contract; the best witness-data target is a derived DAG well-formedness fold. A smaller named-formula interface also simplifies lower calibration. None of these results supplies a 214-state machine.

Evidence boundary. This follow-up independently checks source identities and executes finite Python audits. It does not rerun Lean or Rocq. All new formal drafts written for this follow-up remain *uncompiled*. Recorded upstream builds, mathematical arguments and finite checks are kept separate. Historical finite inputs remain pinned to `beaabc8e3ba9`; production evidence is fixed at review cutoff `0359ebda60aa`, not a moving `main`.

1 What the recent commits actually establish

Phase five is production adoption. Commit `de9fdb0873b3`, reviewed at `0359ebda60aa`, removes the zero offset field from all 2,240 Lean source-certificate entries. The endpoint no longer imports the integer-offset proof. Its shared checker is decoder-parametric, but the concrete K215 decoder remains explicitly 512-step: substituting the legacy 128-step decoder changes 622 entries. All sixteen converted certificate chunks are recorded as freshly checked; regeneration reproduces all eighteen outputs. The source-data file shrinks from 456,548 to 432,131 bytes. This is not yet the 1,426-row quotient.

The combined phase-five receipt records 184 project modules: 37 freshly compiled and 147 predecessor objects reused with source, dependency, object and log bindings. Two typed endpoint clients and ten standard-axiom reports pass with zero audit warnings. A separate compatibility refresh checks fifteen modules and five retained interfaces. These are incremental checks, not a new clean full build. The literal machine and public endpoint statements remain unchanged.

The outer tool session later reported exit 143 after the driver had saved PASS and printed its final completion marker. The cause is not established; no successful outer-shell exit is claimed. Selected compiler processes and both audits returned zero, with subsequent recorded binding reviews. Source logs contain 31 linter warnings, distinct from zero endpoint-audit warnings. This telemetry caveat must accompany the recorded success, not be silently erased or treated as an established kernel failure.

The adopted shared register-simulation cases shorten its main induction from 83 to 61 lines. Shared semantic lemmas remove 56 repeated impossible-tag cases from `InternalSat` and factor bounded-quantifier transport in `ConstructibleAbs`, reducing those files by 112 and five lines. These are already implemented, not further opportunities discovered here.

Earlier placement was not adoption. Production was unchanged through `62f0f95e2b2d`. `beaabc8e3ba9` added our earlier ZIP; `62f0f95e2b2d` placed it as `k215-simplification-3`, without rerunning it. That historical result does not apply to the later phase-five adoption.

The new experiments retain their input meaning. At the review cutoff, the checked runtime files and all fifteen production calibration inputs retain their earlier Git blobs. Although `KSource215Data` changes certificate representation, its complete program/tree/well-formedness

section is byte-identical. `RMEquiv` changes only the proof of `fwd`, not its definitions or statement. These identity checks support the historical finite results; they do not compile the new drafts against phase five.

1.1 The normalized path quotient remains unintegrated

The `k215-simplification/source_normalization` receipt records the exact theorem

$$\forall R : \text{Registers}, \qquad \text{Halts}(p215, ([], R)) \iff \text{Halts}(pathDagPerm, ([], R)).$$

It checks 1,426 entries and 2,851 branches, including the initial branch, with a minimal isolated closure of 22 local modules and eight upstream dependencies. All 22 local sources match successful recorded compile hashes. The receipt describes an incremental isolated Lean 4.32.0 build, not a fresh full machine-to-consistency build. Its source endpoint reports only `propext`, `Classical.choice`, and `Quot.sound`; the generic normalized checker reports `propext` and `Quot.sound`.

The mathematical simplification is precise. With $P \sim Q$ meaning equality of zero-padded path bits, saturate the stored relation by

$$S^*(P, Q) \iff \exists A, B,\ S(A, B) \wedge P \sim A \wedge Q \sim B.$$

The existing observation and segment lemmas turn checked normalized rows into the existing `RMBisim.Bisim`; its halting theorem is reused. Break/continue arithmetic still acts on the *consumed leaf path*, not blindly on a shortened representative. This removes 814 redundant row names without changing a source instruction or TM transition.

Preserve the maintained production checker during integration. Already at `beaabc8e3ba9`, production `RMBisim` had replaced the packet's older `seg_hn` induction by `seg_run` and `hn_after`. Phase five further adds the shared decoder-parametric interface. Adapt the normalized relation to that current infrastructure and retain the explicit 512-step decoder; do not overwrite it with a vendored older module.

1.2 Other checked routes and their limits

The separate `k215-simplification-2/plain_source` packet records a 70-module ordinary 2,240-entry source proof; all 70 source hashes match its receipts. It is not the newly adopted 184-module endpoint closure. The two research closures contain 402,486 source bytes for the 30-module normalized route versus 346,638 for the 70-module plain route. Fewer normalized obligations do not imply smaller source, less memory or faster checking. The normalized packet already uses ordinary bisimulation; removing offsets again is not a second reduction to add to its 1,426 rows.

The Rocq packets provide two useful routes: ordinary bisimulation with a compact path codec and symmetry, and symmetry for the actual offset checker plus compact/original certificate equality. Their recorded Coq 8.18 checks are evidence for those exact sources, not an executed replacement of current production Rocq. The ordinary source theorem compiled, but its separate recursive source-closure `coqchk` audit was not requested; recursive audits of certificate equality and offset symmetry did pass. Neither packet was replayed here under production Rocq 9.1. Preserve the distinction between a compiled source proof and a patch awaiting endpoint integration.

CM215 now has a recorded bridge to the exact repository `KDense215.machine215`. The remaining displayed endpoint adapter is transitivity with the existing K215 theorem. No additional mathematical simulation gap is apparent, but that final adapter still needs compilation and endpoint-assumption checks. CM215 remains a good *incremental* formalization choice, not a demonstrated smaller complete dependency closure.

2 Prove one return to dispatch scan instead of replaying it

The new owned replay finds a common phase consuming most symbolic site-check steps. It starts in state 2, scans left to the outer blank region, then returns to state 84 immediately to the right of the leftmost marker. This is a reusable whole-tape theorem, not a new machine.

Proposition 2.1 (Scanner contract)**.** *Let the current head be at h in state 2. Read leftwards a nonempty finite word W occupying $h, h-1, \ldots, h-|W|+1$. Suppose its last bit is 1, no eight consecutive bits of W are 0, and every cell strictly to its left is 0. The tape to its right is arbitrary. Then the literal K215 table reaches state 84 at head position*

$$h - |W| + 2$$

in exactly $|W| + 18$ concrete steps, with the entire tape unchanged.

The proof reads only 18 table rows. In the chain

$$2, 3, 5, 9, 15, 23, 34, 50,$$

a 1 resets the leftward scan to state 2; a 0 advances the chain, and the eighth consecutive 0 enters state 75. The guard prevents this before the blank exterior. State 75 scans right across nine blank cells, then crosses the marker to state 84. Every traversed bit is rewritten unchanged. This yields $|W| + 8 + 10$ steps and the displayed head location. It also supplies strictly positive progress for composition.

Symbolic interface. In a zipper whose left list is read from the head outwards, write that list as W followed by blank padding, and its arbitrary right continuation as R. The macro result has marker 1 as its singleton left list and

$$\mathrm{reverse}(\mathrm{init}(W)) +\!\!+ R$$

as its right list, with the head on the right side. Absolute-coordinate translation is part of the contract. Opaque protected tails are carried as a frame, not inspected or erased.

The executable guard handles affine repeated runs conservatively. Unbounded zero runs inside W are rejected. A true run which can vanish at parameter zero must not be treated as a guaranteed separator: it may join two zero streaks. Two genuine site cases fail this sufficient guard and correctly fall back to ordinary steps. A sufficient guard need not be complete.

Owned replay quantity	Count
Guarded site cases completed	6,176
Original symbolic steps	2,442,801
Accepted scanner macro uses	15,673
Original steps within those scanner phases	1,532,123
Accelerated ordinary steps plus macro calls	926,351
Abstract symbolic steps removed	1,516,450 (62.08%)

This is not merely subtraction from a profile. A separate accelerated runner resumes from the macro's computed result and reaches every original guarded target. Whole-tape finite checks cover 32,444 short-word cases and 1,485 affine instantiations; a separate mathematical review adds independently implemented tape checks. These are regression evidence, not a universal proof or a benchmark of Lean reduction. The percentage excludes guard cost and does not claim elapsed-time, memory, or machine-state savings.

Kernel gate. Prove the scanner induction and full-tape frame first; then prove symbolic instantiation of the guard, the zipper result and positive progress. Refine the existing checker with this macro while retaining its original fallback and public site theorem. Benchmark this together with the already-proved fast table lookup, rather than changing semantics or trusting a Python jump.

3 Factor decoder roles without erasing their guards

An incoming proposal covers 4,750 expanded internal source occurrences by 234 decoder roles. Those roles distinguish the source node, physical state and a short preceding-bit guard. Some duplicate the same node/state with different allowed guards. Grouping by *node and physical state*, while retaining the union of guards, gives 180 roles. This is a proof-certificate quotient, not a merger of TM states.

The independent checker binds the exact expanded source tree and literal machine rows. It checks 287 internal bindings, 73 primitive bindings and 371 guarded local cases. There are 281 distinct local trace contracts, a count already present in the incoming packet: 252 need no preceding-bit read, 25 read one bit, and four read two. The root is fixed to the source root, state 84 and guard 1. Every internal transition preserves its local tape and carries an allowed guard to its child; primitive exits retain their literal action and state.

The natural formal structure is a local contract library indexed by state, current bit and the actually required suffix, followed by a small structural composition theorem. Keep the source-node binding separate from that shared physical contract. A suffix is an invariant hypothesis, not decorative certificate data: merging states or dropping suffix checks would admit different control behavior. Negative controls reject a changed root state, changed root guard, malformed index and missing role. The verbose factored JSON is larger, so no storage saving is claimed.

A primitive binding proves only its first transition, not the full primitive/return contract. That missing theorem must quantify the whole path and require membership in the role graph's accepted prefix language, in addition to the original gap, branch and register conditions. The same two-bit suffix alone is not enough: a primitive can revisit dispatch with modified path data. Thus 180 guarded interfaces do not replace the 6,176 complete runtime obligations.

This is complementary to the scanner macro. The scanner compresses repeated long physical returns; the guarded decoder library shares the small local dispatch proofs. Begin with the scanner, whose ordinary proof is particularly small. Only then measure whether the 180-role quotient improves the remaining checker enough to justify its additional data structure. The guarded-composition theorem and its connection to every site endpoint are still kernel obligations.

4 Derive well formedness witnesses from the DAG

The checked source lanes still provide three lists H, B, C of length 487 and verify inequalities against them. These 1,461 supplied numbers describe structure already present in the graph. A memoized all-node fold can compute the escape bounds and obtain the height witness from completion order.

For an edge e, let B_e, C_e be its escaping break/continue levels. A decrement has $(B_e, C_e) = (1, 0)$; break k has $(k, 0)$; continue k has $(0, k)$; increment and halt have $(0, 0)$. A node edge reads its previously completed child's values. If node n has car edge a and cdr edge b, define

$$B_n = \max(B_a \dot{-} 1, B_b),$$
$$C_n = \max(C_a, C_b \dot{-} 1),$$

where $x \dot{-} 1 = \max(x - 1, 0)$ on naturals. These are the least values satisfying the existing four escape inequalities. The current generator already computes these recurrences in Python; the new proposal moves that construction under one proved evaluator. The asymmetry matters: car consumes one break level; cdr consumes one continue level. Replacing both by an undirected maximum is wrong.

Use unseen/active/complete marks to reject cycles and memoize shared nodes. Let $\rho(n)$ be the positive completion index of node n. Each node child completes first, hence

$$\rho(\text{child}) < \rho(n), \qquad 1 \le \rho(n) \le N.$$

With primitive height zero, $H(n) := \rho(n)$ is already a witness for the existing `RMEquiv.WF.height`; the root bound $H(r) \le N + 1$ follows. There is no need to calculate or transmit a minimal-height array merely to prove acyclicity and sufficient decoder fuel.

Acceptance conditions must stay exact. Require $r < N$, every node reference below N, positive break/continue levels, completion of *every* bounded node, and $B_r = C_r = 0$. The current `WF` quantifies over every bounded node, not only reachable nodes. The target graph has 261 reachable nodes among 487; a cycle in one of the other 226 must still be rejected. Register-index bounds are not part of this particular `WF` and must not be silently added.

For the literal 487-node graph, the computed canonical arrays exactly match all supplied H, B, C entries; the root is $(H, B, C) = (178, 0, 0)$. Completion rank also satisfies the original inequalities, with root rank 261. The iterative implementation uses 1,859 jobs against its $4N = 1,948$ structural work bound. An independent repeated-readiness implementation agrees on all 14,741 enumerated one- and two-node graphs, and every accepted graph satisfies the original inequalities. Eleven adversarial controls include unreachable cycles and escaping break/continue levels.

The intended simplification removes *supplied* summaries and their redundant consistency checks, not all witness computation. The normalized packet has 16 well-formedness checks within one program module; only the `k215-simplification-2/plain_source` lane puts them in 16 separate files. Do not report sixteen file deletions for both lanes.

Kernel gate. Use a total, fuel-bounded evaluator which rejects on exhaustion. Prove completed-table invariants and soundness for arbitrary fuel, retaining a final scalar check $H(r) \le N + 1$; a formal completeness proof or $4N$ counting theorem is not needed for this first version. Derive `WF` from acceptance and evaluate the literal instance. One generic soundness theorem can replace the hand-supplied arrays while preserving the existing structured/path-semantics theorem. If kernel reduction of the fold is expensive, keep transparent chunked execution or checked intermediate states; an elegant algorithm is not automatically the cheapest kernel computation.

5 Narrow the lower calibration to the named seeds it uses

The earlier semantic round-trip proposal is still useful, but it unnecessarily enters the numerical decoder to recover a seed already expressed in named syntax. Two smaller changes expose a cleaner lower-bound interface.

5.1 Fresh universal padding reuses an existing theorem

The repository already proves `Code254GIC.occurs_lt_enc`, including binder occurrences, and `not_free_enc`. No new freshness induction is required. For any named formula ψ, set

$$\mathrm{pad}(\psi) := \forall_{\mathrm{enc}(\psi)}\psi.$$

Its binder is not free in ψ. Therefore, for every membership relation and displayed valuation e,

$$\mathrm{Sat}(\mathrm{mem}, e, \mathrm{pad}(\psi)) \iff \mathrm{Sat}(\mathrm{mem}, e, \psi).$$

Forward elimination uses the explicit witness $e(\mathrm{enc}\,\psi)$; reverse introduction uses freshness and valuation extensionality. The proof needs neither irreflexivity nor equality-freeness and does not infer an inhabitant of an empty domain. Padding is constructive, preserves equality-freeness definitionally, and its encoding exceeds 1. It adds one formula constructor rather than the four added by literal double negation; this is a local syntax saving, not a measured build improvement.

Define a sparse named axiom family

$$\mathrm{SeedAx}(f) \iff \exists\psi,\ \mathrm{EqFree}(\psi) \wedge f = \mathrm{bundleNF}(0, 1, \mathrm{enc}(\mathrm{pad}\,\psi), \mathrm{pad}\,\psi).$$

Every such formula is derivable by the existing bundle rule, so it can feed the unchanged `KEind222.c222_complete`. The lower endpoint need not use the theory of *all* derivable formulas to obtain the seeds it actually consumes. This narrows a model interface, not the machine's axiom enumerator: the inherited logical-completeness infrastructure and AX heads are not thereby removable. The supplied first-stage draft implements this adapter, but still uses two semantic encode/decode equivalences. It is uncompiled.

5.2 The next named bundle theorem removes those two decoder uses

Let

$$R(x, y) := \mathrm{Sat}(\mathrm{mem}, e[a \mapsto x][b \mapsto y], m).$$

The precise remaining theorem is

$$\begin{aligned} &a \neq b, \quad k \neq a, \quad k \neq b, \quad k \notin \mathrm{Free}(m) \\ &\quad \Longrightarrow \quad \mathrm{Sat}(\mathrm{mem}, e, \mathrm{bundleNF}(a, b, k, m)) \iff \mathrm{SeedProp}(\mathrm{mem}, R, e(a)). \end{aligned}$$

Prove it by named request semantics, named closure semantics and freshness-based environment transport. It needs no irreflexivity or equality-free premise for *semantic normalization*; equality-freeness remains necessary separately for the code-calculus derivability bridge. Its existential normalization is classical, unlike padding itself.

At roles $a = 0, b = 1$, use $m = \mathrm{pad}\,\psi$ and $k = \mathrm{enc}\,m$. Existing freshness and $1 < k$ discharge the side conditions. Then `seedProp_congr` and padding give the required seed directly. This would remove the raw decoder from this lower semantic path, not automatically from module imports or from the entire proof.

Do not overgeneralize the interface. The upper soundness direction still covers *every natural code*, including junk codes and coincident ordinary roles. In particular, code 1 decodes to the self-membership atom rather than genuine bottom, so its semantic falsity still uses irreflexivity. Preserve raw `bundle_of_seed`, the equality interpretation, the indiscernibility quotient and all exceptional-role cases. This changes neither the consistency strength nor the literal machine, and no proof-size or runtime improvement is yet measured.

6 Allow finite terminal cleanup in the transport API

The new positive-stuttering source also reveals a small genuine weakening of its hypotheses. Let A, B be deterministic *partial* transition systems, with successor functions into an option type. A terminal configuration has no successor. Let R relate configurations and assume:

1. If $R(a, b)$ and $a \to a'$, there are $k > 0, b'$ with a successful target run $b \to^k b'$ and $R(a', b')$.
2. If $R(a, b)$ and a is terminal, then b eventually halts.

Then $R(a, b)$ implies $\mathrm{Halts}(a) \leftrightarrow \mathrm{Halts}(b)$. Immediate terminal agreement is stronger than necessary.

Preservation follows by induction on a terminating source run, using the second premise only in its base case. Reflection uses the first premise alone: strongly induct on a terminating target run length. If the source is nonterminal, its successful positive target block is a prefix of that deterministic terminating run. It cannot pass a terminal configuration. Removing it strictly decreases the remaining length.

The existing strict interface implies the weaker terminal-drain premise, so it can remain as a compatibility constructor. The inspected reflection proof does not use its terminal field at all. The derivative draft changes that field and the preservation base case; it does not overwrite production code.

This permits a terminal source to correspond to a target that performs finite cleanup before halting. The finite audit exhausts 36,864 small systems/relations; 7,273 satisfy both new hypotheses, including 1,506 that do not satisfy strict terminal agreement. Zero-length progress and missing terminal drain have explicit counterexamples. *Do not transfer this proof unchanged to a total step function with an absorbing halt*: positive absorbing steps would destroy the reflection argument. This is an API simplification, not a new CM215 invariant or state reduction.

7 Ranked integration plan and acceptance gates

1. **Build on phase five rather than repeat it.** Ordinary 2,240-row Lean checking is now production. Adapt the separately checked 1,426-row quotient to the current decoder-parametric checker and 512-step decoder; preserve the public theorem and rebuild endpoint clients and axiom reports. In parallel integrate the compact single-certificate Rocq route. Do not confuse either route with packet intake.
2. **Prove the scanner macro and refine the site checker.** Its 18-row ordinary argument and 6,176-case replay give the clearest new cost target. Preserve full tape, symbolic guards, positive progress and fallback. Measure clean and incremental checking time and peak memory.
3. **Replace supplied DAG summaries with a sound fold.** Reuse `RMEquiv.WF`; cover all bounded nodes and both asymmetric escape rules. Compare one fold with transparent batched checking before removing old witnesses.
4. **Build the named lower-calibration interface.** Compile padding and sparse seeds, then prove the named bundle theorem and remove only the now-unused lower decoder path. Keep raw upper soundness and the exact endpoint statement.
5. **Add the guarded decoder library and terminal-drain API when useful.** Both have precise contracts and finite regression tests, but their integration benefit should be demonstrated rather than assumed.

For every accepted change, retain exact table hashes, bind source receipts to the rebuilt source, reject placeholder/native-evaluator assumptions, and test the final public machine-to-consistency theorem. No 214-state construction is established by this report.

8 Pins and reproducibility

Historical proposal and finite-audit inputs are pinned to `beaabc8e3ba9`; phase-five production evidence is separately pinned to `0359ebda60aa`. Neither means an unqualified moving `main`. Full paths, Git blob IDs and retrieved-source hashes are in the accompanying per-topic `SOURCE_PINS.json` or `INPUT_PINS.json`. The important commit and tree identities are:

- Review base: `557b6aac99b836a8ab61fb210ba9d59061586202`.
- Historical finite/proposal pin: `beaabc8e3ba961a8892d35a0683c2ed52e4ab4dc`.
- Placement-only cutoff: `62f0f95e2b2df5b380ae502eb65bd64f1ee90209`.
- Production adoption: `de9fdb0873b36d049d075a5c0568f2dfcff28e7a`.
- Review cutoff: `0359ebda60aaf52758358e1450a04988136db67e`.
- Normalized receipt's original build pin: `b6c7087477e51c017003bd729e712c8969733223`.

The pinned proposal directory contains the upstream source-normalization receipt and proof packets. The pinned freshness lemmas are existing production work, not new results of this follow-up.

Per-topic source pins, independent checkers and uncompiled drafts accompany this report. The pinned phase-five combined-check account records the incremental and outer-session limits. Build with `report/build.sh`; final PDF rendering and inspection are recorded in `report/qa/QA.json`.